

4.2.2(b,d,f): Question

b) $S \rightarrow +SS \mid *SS \mid a$
with string $+*aaa$

d) $S \rightarrow S+S \mid SS \mid (S) \mid S* \mid a$
with string $(a+a)*a$

f) $S \rightarrow aSbS \mid bSaS \mid \epsilon$
with string $aabbab$

a) Give a left-most derivation for the string.

b) Give a rightmost derivation for the string.

c) Give a parse tree for the string.

Answer:

(b)

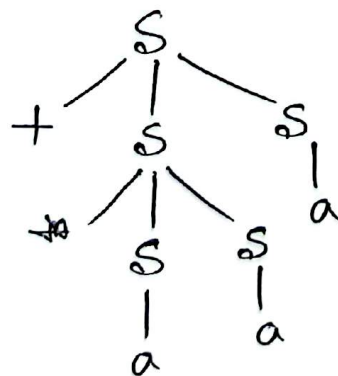
Left-most derivation :

$S \rightarrow +SS$ ($S \rightarrow +SS$)
 $\rightarrow +*SSS$ ($S \rightarrow *SS$)
 $\rightarrow +*aSS$ ($S \rightarrow a$)
 $\rightarrow +*aaS$ ($S \rightarrow a$)
 $\rightarrow +*aaa$ ($S \rightarrow a$)

Right-most derivation:

$S \rightarrow +SS$ ($S \rightarrow +SS$)
 $\rightarrow +Sa$ ($S \rightarrow a$)
 $\rightarrow +*SSa$ ($S \rightarrow *SS$)
 $\rightarrow +*Saa$ ($S \rightarrow a$)
 $\rightarrow +*aaa$ ($S \rightarrow a$)

Parse tree:



Derived string: $+*aaa$

(d)

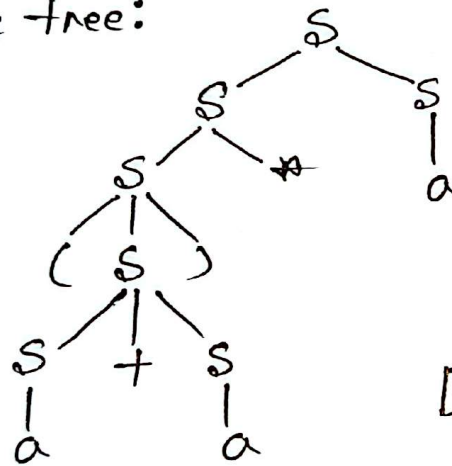
Left-most derivation:

$$\begin{aligned} S &\rightarrow SS \quad (S \rightarrow SS) \\ &\rightarrow S * S \quad (S \rightarrow S *) \\ &\rightarrow (S) * S \quad (S \rightarrow (S)) \\ &\rightarrow (S + S) * S \quad (S \rightarrow S + S) \\ &\rightarrow (a + S) * S \quad (S \rightarrow a) \\ &\rightarrow (a + a) * S \quad (S \rightarrow a) \\ &\rightarrow (a + a) * a \quad (S \rightarrow a) \end{aligned}$$

Right-most derivation:

$$\begin{aligned} S &\rightarrow SS \quad (S \rightarrow SS) \\ &\rightarrow Sa \quad (S \rightarrow a) \\ &\rightarrow S * a \quad (S \rightarrow S *) \\ &\rightarrow (S) * a \quad (S \rightarrow (S)) \\ &\rightarrow (S + S) * a \quad (S \rightarrow S + S) \\ &\rightarrow (S + a) * a \quad (S \rightarrow a) \\ &\rightarrow (a + a) * a \quad (S \rightarrow a) \end{aligned}$$

Parse tree:



Derived string: $(a + a) * a$

(e)

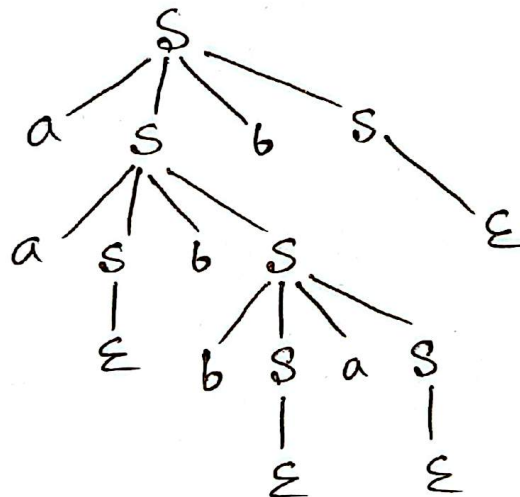
Left-most derivation:

$$\begin{aligned} S &\rightarrow aSbS \quad (S \rightarrow aSbS) \\ &\rightarrow aaSbSbS \quad (S \rightarrow aSbS) \\ &\rightarrow aabSbS \quad (S \rightarrow \epsilon) \\ &\rightarrow aabbSaSbS \quad (S \rightarrow bSaS) \\ &\rightarrow aabbasbS \quad (S \rightarrow \epsilon) \\ &\rightarrow aabbabS \quad (S \rightarrow \epsilon) \\ &\rightarrow aabbab \quad (S \rightarrow \epsilon) \end{aligned}$$

Right most derivation:

$S \rightarrow aSbS$ ($S \rightarrow aSbS$)
 $\rightarrow aSb$ ($S \rightarrow \epsilon$)
 $\rightarrow aaSbSb$ ($S \rightarrow aSbS$)
 $\rightarrow aaSbbsaSb$ ($S \rightarrow bSas$)
 $\rightarrow aaSbbsab$ ($S \rightarrow \epsilon$)
 $\rightarrow aaSbbsab$ ($S \rightarrow \epsilon$)
 $\rightarrow aabbab$ ($S \rightarrow \epsilon$)

Parse tree:



Derived string: aabbab

4.3.2 (a, b, c): Question

- a) $S \rightarrow SS+ | SS* | a$
with string $aa+a*$
- b) $S \rightarrow OS1 | O1$
with string 000111
- c) $S \rightarrow S(S)S | \epsilon$
with string $(())$.

- a) Left factor this grammar
- b) Does left factoring make the grammar suitable for top-down parsing?
- c) In addition to left factoring, eliminate the left recursion from original grammar
- d) Is the resulting grammar suitable for top-down parsing?

Answer:

(a) $\rightarrow$ (a) $S \rightarrow SSS' | a$
 $S' \rightarrow + | *$ which is the left-factored grammar

(b) No, left factoring the given grammar doesn't make the grammar suitable for top-down parsing, as there is left recursion in the production.

(c) Removing left recursion:

$$S \rightarrow aS''$$

$$S'' \rightarrow SS'S'' | \epsilon$$

$$S' \rightarrow + | *$$

(d) Resulting grammar is suitable now (from 'c') for top-down parsing as non-determinism is removed by factoring and after that left recursion is removed which cause ambiguity in top-down parsing. 4

(b) $\rightarrow$ (a)

$S \rightarrow OS'$

$S' \rightarrow S1|1$; which is the left-factored grammar.

(b) Removal of non-determinism through left factoring makes this grammar suitable for top-down parsing ; as there is also no left-recursion.

(c) As, there is no left recursion, elimination process is not required.

(d) Resulting grammar is suitable for top-down parsing, as there are no non-determinism and no left recursion.

(c) $\rightarrow$

(a) Left factoring is not required as no non-determinism.

(b) The grammar is not suitable for top-down parsing as there is left recursion.

(c) Removing left-recursion:

$S \rightarrow S'$

$S' \rightarrow (S)SS' | \epsilon$

(d) As, the grammar is deterministic after removing left-recursion, it is suitable now for top-down parsing.

4.4.3: Question

Compute first and follow for the grammar

$$S \rightarrow SS+ | SS* | a$$

Answer: Left factoring:

$$S \rightarrow SSS' | a$$

$$S' \rightarrow + | *$$

Removing left-recursion:

$$S \rightarrow aS''$$

$$S'' \rightarrow SS'S'' | \epsilon$$

$$S' \rightarrow + | *$$

Productions	FIRST()	FOLLOW()
$S \rightarrow aS''$	$\{a\}$	$\{\$, +, *\}$
$S' \rightarrow + *$	$\{+, *\}$	$\{a, \$, +, *\}$
$S'' \rightarrow SS'S'' \epsilon$	$\{a, \epsilon\}$	$\{\$, +, *\}$

4.4.4: Question

Compute first and follow for each of the following grammars:

a) $S \rightarrow 0S1 | 01$

b) $S \rightarrow +SS | *SS | a$

c) $S \rightarrow S(S)S | \epsilon$

d) $S \rightarrow S+S | SS | (S) | S* | a$

e) $S \rightarrow (L) | a$ and $L \rightarrow L, S | S$

f) $S \rightarrow aSbS | bSaS | \epsilon$

(a) Given grammar:
 $S \rightarrow 0S1 \mid 01$

Left factoring:

$$S \rightarrow 0S'$$

$$S' \rightarrow S1 \mid 1$$

Productions	FIRST()	FOLLOW()
$S \rightarrow 0S'$	{0}	{\$, 1}
$S' \rightarrow S1 \mid 1$	{0, 1}	{\$, 1}

(b) Given grammar:

$$S \rightarrow +SS \mid *SS \mid a$$

Productions	FIRST()	FOLLOW()
$S \rightarrow +SS \mid *SS \mid a$	{+, *, a}	{\$, +, *, a}

(c) Given grammar:

$$S \rightarrow S(S)S \mid \epsilon$$

Removing left recursion:

$$S \rightarrow S'$$

$$S' \rightarrow (S)SS' \mid \epsilon$$

Productions	FIRST()	FOLLOW()
$S \rightarrow S'$	{(, ε}	{\$,), (}
$S' \rightarrow (S)SS' \mid \epsilon$	{(, ε}	{\$,), (}

(d) Given grammar:

$$S \rightarrow S+S | SS | (S) | S* | a$$

Left factoring:

$$S \rightarrow SX | (S) | a$$

$$X \rightarrow +S | S | *$$

Removing left recursion:

$$S \rightarrow (S)S' | aS'$$

$$S' \rightarrow XS' | \epsilon$$

$$X \rightarrow +S | S | *$$

Productions	FIRST()	FOLLOW()
$S \rightarrow (S)S' aS'$	$\{ (, a \}$	$\{ +, (, a, *, \$,) \}$
$S' \rightarrow XS' \epsilon$	$\{ +, (, a, *, \epsilon \}$	$\{ +, (, a, *, \$,) \}$
$X \rightarrow +S S *$	$\{ +, (, a, * \}$	$\{ +, (, a, *, \$,) \}$

(e) Given grammar:

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

Removing left recursion,

$$S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow ,SL' | \epsilon$$

Productions	FIRST()	FOLLOW()
$S \rightarrow (L) a$	$\{ (, a \}$	$\{ \$, ,,) \}$
$L \rightarrow SL'$	$\{ (, a \}$	$\{) \}$
$L' \rightarrow ,SL' \epsilon$	$\{ ,, \epsilon \}$	$\{) \}$

(f) $S \rightarrow aSbS | bSaS | \epsilon$

Productions	FIRST()	FOLLOW()
$S \rightarrow aSbS bSaS \epsilon$	$\{ a, b, \epsilon \}$	$\{ \$, b, a \}$

5.1.1 (a) Question:

For the SDD of fig. 5.1, give annotated parse tree for the following expression:

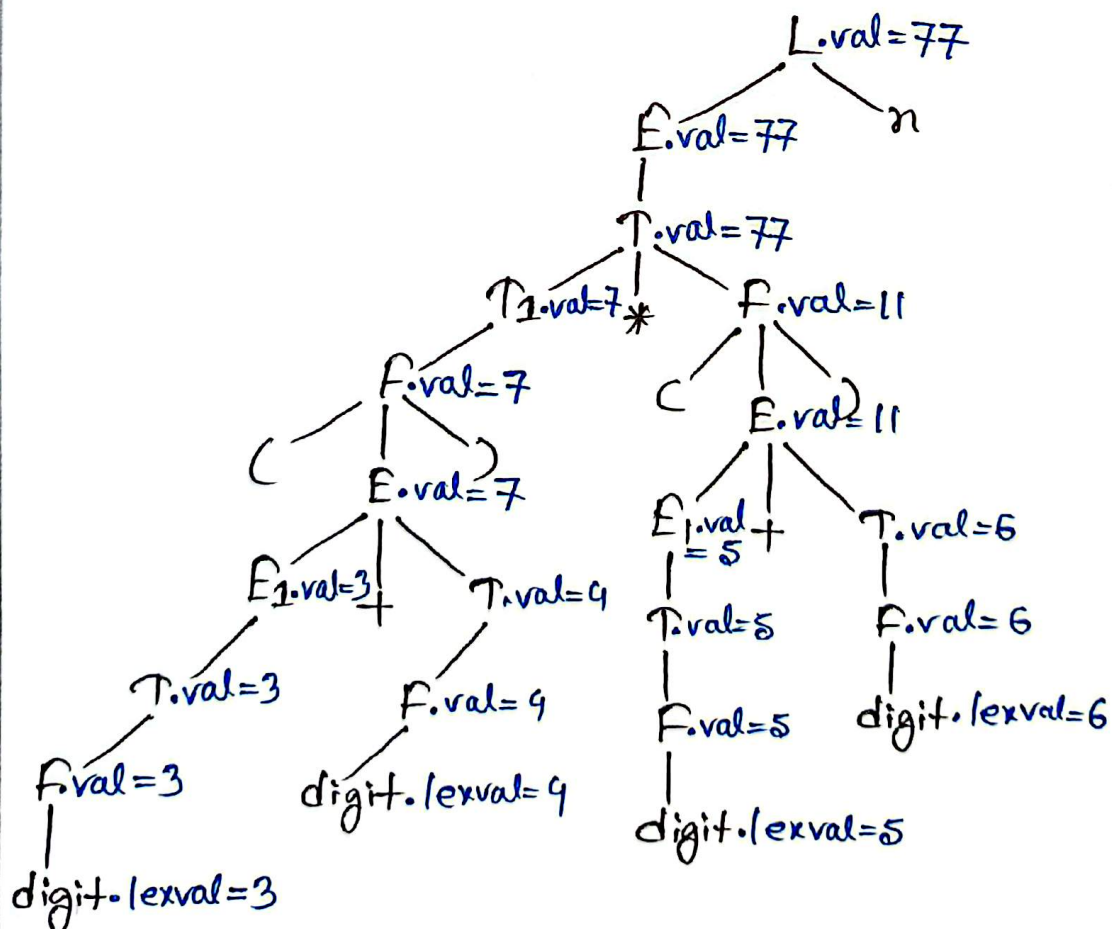
a) $(3+4) * (5+6) n$.

b) $1 * 2 * 3 * (4+5) n$.

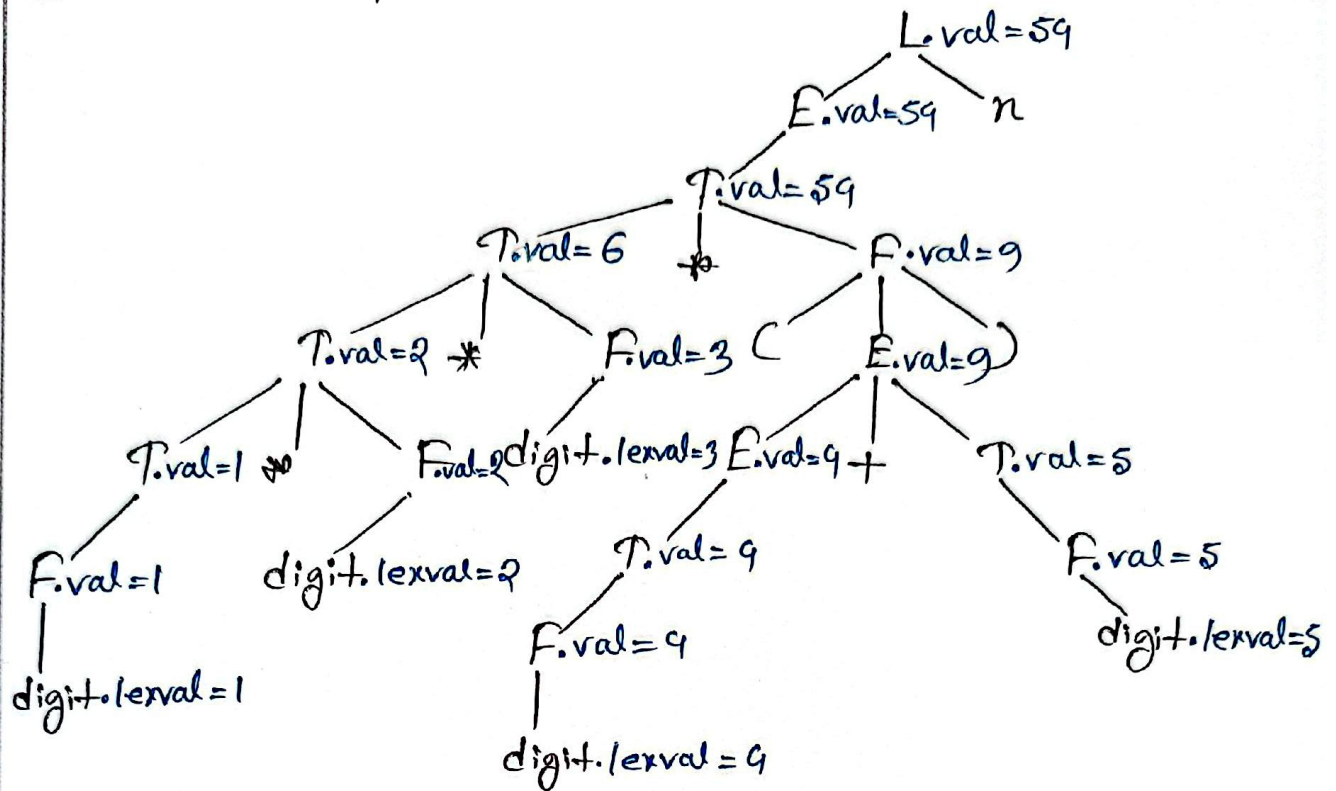
Production	Semantic rules
1) $L \rightarrow En$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Figure 5.1

Answer: (a) Annotated Parse Tree for $(3+4) * (5+6) n$



(b) Annotated parse tree for : $1 * 2 * 3 * (4 + 5) n$.



4.5.3: Question:

Give bottom-up parses for the following input strings and grammars:

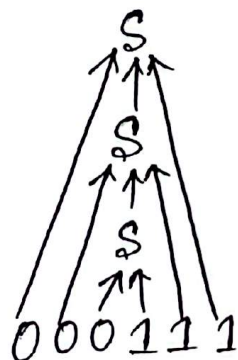
(a) Grammar: $S \rightarrow 0S1 \mid 01$
Input : 000111

(b) Grammar: $S \rightarrow SS+ \mid SS* \mid a$
Input : $aaa * a++$

Answer:

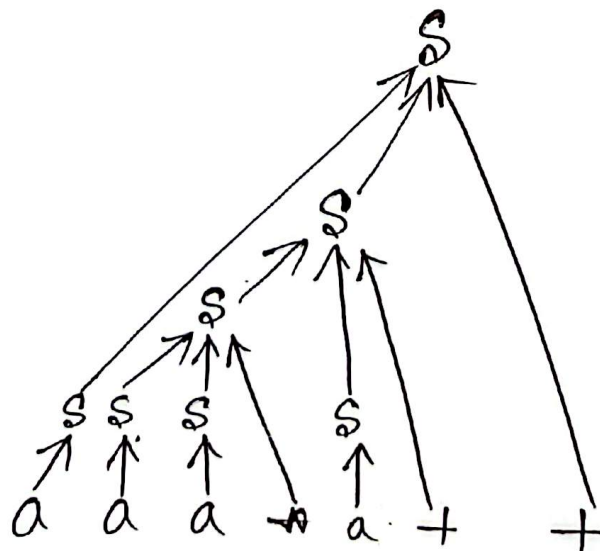
(a)

Stack	Input	Action
\$	000111\$	Shift
\$0	00111\$	Shift
\$00	0111\$	Shift
\$000	111\$	Shift
\$0001	11\$	Reduce by: $S \rightarrow 01$
\$00S	11\$	Shift
\$00S1	1\$	Reduce by: $S \rightarrow 0S1$
\$0S	1\$	Shift
\$0S1	\$	Reduce by: $S \rightarrow 0S1$
\$S	\$	Accept



(b)

Stack	Input	Action
\$	aaa*a++\$	Shift
\$a	aa*a++\$	Reduce by: $S \rightarrow a$
\$S	aa*a++\$	Shift
\$Sa	a*a++\$	Reduce by: $S \rightarrow a$
\$SS	a*a++\$	Shift
\$SSa	*a++\$	Reduce by: $S \rightarrow a$
\$SSS	*a++\$	Shift
\$SSS*	a++\$	Reduce by: $S \rightarrow SS*$
\$SS	a++\$	Shift
\$SSa	++\$	Reduce by: $S \rightarrow a$
\$SSS	++\$	Shift
\$SSS+	+\$	Reduce by: $S \rightarrow SS+$
\$SS	+\$	Shift
\$SS+	\$	Reduce by: $S \rightarrow SS+$
\$S	\$	Accept



6.1.1 Construct the DAG for the expression:

$$((x+y) - ((x+y) * (x-y))) + ((x+y) * (x-y))$$

Answer:

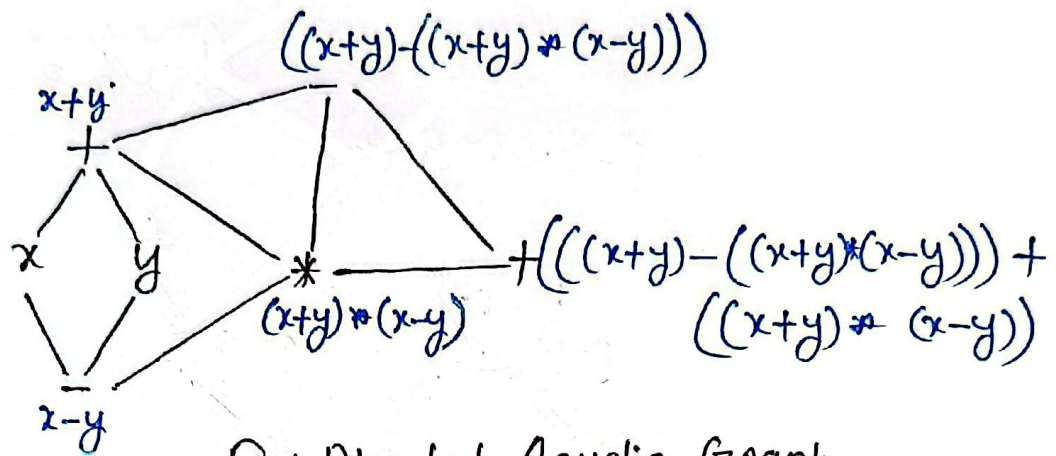


Fig: Directed Acyclic Graph for given expression.

6.2.1. Translate the arithmetic expression $a + -(b+c)$

into:

a) A syntax tree.

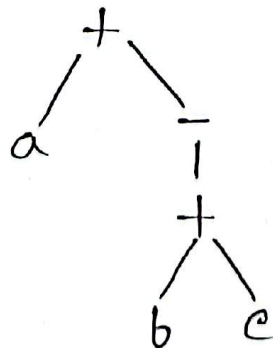
b) Quadruples.

c) Triples.

d) Indirect triples.

Answer:

(a) Syntax tree for given expression $a + -(b+c)$



(b) Given expression:

$$a + -(b + c)$$

Three address code for the given expression:

$$t_1 = b + c$$

$$t_2 = \text{uminus } t_1$$

$$t_3 = a + t_2$$

Quadruples:

Location	op	arg1	arg2	Result
(0)	+	b	c	t ₁
(1)	-	t ₁		t ₂
(2)	+	a	t ₂	t ₃

(c) Triples:

Location	op	arg1	arg2
(0)	+	b	c
(1)	-	(0)	
(2)	+	a	(1)

(d) Indirect triples:

Pointers	Statement
35	(0)
36	(1)
37	(2)

8.2.1. Generate code for the following three-address statements assuming all variables are stored in memory locations.

c) $x = a + 1$

d) $x = a + b$

e) The two statements

$x = b * c$

$y = a + x$

Answer:

(c) $x = a + 1$

Target code:

LD R0, a

ADD R0, R0, 1

ST x, R0

(d) $x = a + b$

Target code:

LD R0, a

LD R1, b

ADD R0, R0, R1

ST x, R0

(e) $x = b * c$

$y = a + x$

Target code:

LD R0, b

LD R1, c

MUL R0, R0, R1

ST x, R0

LD R1, a

ADD R1, R1, R0

ST y, R1

8.2.2: Generate code for the following three-address statements assuming a and b are arrays whose elements are 4-byte values.

(a) The four-statement sequence

$$\begin{aligned}x &= a[i] \\ y &= b[j] \\ a[i] &= y \\ b[j] &= x\end{aligned}$$

(b) The three-statement sequence

$$\begin{aligned}x &= a[i] \\ y &= b[j] \\ z &= x * y\end{aligned}$$

Answer:

(a) $\text{address} = \text{base} + \text{index} \times 4$

Generated code:

```
LD R0, i
MUL R0, R0, 4
LD R1, a(R0)
ST x, R1

LD R2, j
MUL R2, R2, 4
LD R3, b(R2)
ST y, R3

ST a(R0), R3
ST b(R2), R1
```

(b) $\text{address} = \text{base} + \text{index} \times 4$

LD R0, i

MUL R0, R0, 4

LD R1, a(R0)

ST x, R1

LD R2, b(R0)

ST y, R2

MUL R3, R1, R2

ST z, R3

8.2.4 Generate code for the following sequence assuming that x, y, z are in memory locations:

if $x < y$ goto L1

z = 0

goto L2

L1: z = 1

Answer: Generated code:

LD R0, x

LD R1, y

CMP R0, R1

JL L1

MOV R0, 0

ST z, R0

JMP L2

L1:

MOV R0, 1

ST z, R0

L2:

Exercise 8.2.6: Determine the costs of the following instruction sequences: of a), b) and c)

(a)

Instructions	Cost/ Instruction	Total cost
LD R0, y	$1+1=2$	Total = $2+2+1$ $2=7$
LD R1, z	$1+1=2$	
ADD R0, R0, R1	$1+0=1$	
ST x, R0	$1+1=2$	

(b)

Instructions	Cost/ Instruction	Total Cost
LD R0, i	$1+1=2$	Total = $2+2+2$ $+2=8$
MUL R0, R0, 8	$1+1=2$	
LD R1, a(R0)	$1+1=2$	
ST b, R1	$1+1=2$	

(c)

Instructions	Cost/ Instruction	Total Cost
LD R0, c	$1+1=2$	Total = $2+2+2$ $+2=8$
LD R1, i	$1+1=2$	
MUL R1, R1, 8	$1+1=2$	
ST a(R1), R0	$1+1=2$	